

Name : Srushti Patil
Roll No.: C-04
Software Proficiency Program II (Python Programming)

Experiment - 13

1. Create a CSV file named employees.csv with columns Name, Age, Department, Salary. Load the data, filter employees older than 30, group by Department, and calculate the average, minimum, and maximum salary for each department.

main.py	Output
<pre>1 # Exp-13: Q1 - Employees DataFrame Analysis (No CSV file needed) 2 # Create DataFrame, filter, group, and perform aggregation in memory 3 4 import pandas as pd 5 6 # Step 1: Create sample employee data 7 employees = pd.DataFrame({ 8 "Name": ["Alice", "Bob", "Charlie", "David", "Eva", "Frank", "Grace", 9 "Henry"], 10 "Age": [25, 32, 45, 28, 36, 41, 29, 39], 11 "Department": ["HR", "IT", "Finance", "HR", "IT", "Finance", "IT", "HR"], 12 "Salary": [40000, 55000, 75000, 42000, 65000, 80000, 60000, 47000] 13 }) 14 print("=== Employees Data ===") 15 print(employees) 16 17 # Step 2: Filter employees older than 30 18 filtered = employees[employees["Age"] > 30] 19 print("\n=== Employees older than 30 ===") 20 print(filtered) 21 22 # Step 3: Group by Department and calculate average, min, and max salary 23 salary_stats = filtered.groupby("Department")["Salary"].agg(["mean", "min", 24 "max"]).reset_index() 25 print("\n=== Salary Stats by Department ===") 26 print(salary_stats)</pre>	<pre>=== Employees Data === Name Age Department Salary 0 Alice 25 HR 40000 1 Bob 32 IT 55000 2 Charlie 45 Finance 75000 3 David 28 HR 42000 4 Eva 36 IT 65000 5 Frank 41 Finance 80000 6 Grace 29 IT 60000 7 Henry 39 HR 47000 === Employees older than 30 === Name Age Department Salary 1 Bob 32 IT 55000 2 Charlie 45 Finance 75000 4 Eva 36 IT 65000 5 Frank 41 Finance 80000 7 Henry 39 HR 47000 === Salary Stats by Department === Department mean min max 0 Finance 77500.0 75000 80000 1 HR 47000.0 47000 47000 2 IT 60000.0 55000 65000 === Code Execution Successful ===</pre>

2. Create a CSV file named sales.csv with columns SaleID, Product, Amount, Customer. Load the data, filter sales with amount greater than 50,000, group by Product, and calculate total and average sales per product.

main.py	Output
<pre>1 # Exp-13 Q2: Sales Analysis (No pandas) 2 from collections import defaultdict 3 sales = [4 {"SaleID":101, "Product":"Laptop", "Amount":75000, "Customer":"A"}, 5 {"SaleID":102, "Product":"Mouse", "Amount":2000, "Customer":"B"}, 6 {"SaleID":103, "Product":"Keyboard", "Amount":50000, "Customer":"C"}, 7 {"SaleID":104, "Product":"Laptop", "Amount":120000, "Customer":"D"}, 8 {"SaleID":105, "Product":"Mouse", "Amount":60000, "Customer":"E"} 9] 10 # Filter sales > 50000 11 filtered = [s for s in sales if s["Amount"] > 50000] 12 # Group by Product and calculate total and average 13 grouped = defaultdict(list) 14 for s in filtered: 15 grouped[s["Product"]].append(s["Amount"]) 16 stats = {p: {"total": sum(v), "average": sum(v)/len(v)} for p,v in grouped.items() 17 } 18 print("Sales > 50,000:") 19 for s in filtered: 20 print(s) 21 print("\nTotal & Average Sales per Product:") 22 for prod, stat in stats.items(): 23 print(prod, stat)</pre>	<pre>Sales > 50,000: {'SaleID': 101, 'Product': 'Laptop', 'Amount': 75000, 'Customer': 'A'} {'SaleID': 104, 'Product': 'Laptop', 'Amount': 120000, 'Customer': 'D'} {'SaleID': 105, 'Product': 'Mouse', 'Amount': 60000, 'Customer': 'E'} Total & Average Sales per Product: Laptop {'total': 195000, 'average': 97500.0} Mouse {'total': 60000, 'average': 60000.0} === Code Execution Successful ===</pre>

Name : Srushti Patil

Roll No.: C-04

Software Proficiency Program II (Python Programming)

3. Create a CSV file named **student marks.csv** with columns **StudentID**, **Name**, **Maths**, **Science**, **English**. Load the data, filter students with Maths marks above 80, group by student name, and calculate total and average marks per student.

main.py	Output
<pre>1 # Exp-13 Q3: Student Marks Analysis (No pandas) 2 students = [3 {"StudentID":1,"Name":"Alice","Maths":85,"Science":80,"English":78}, 4 {"StudentID":2,"Name":"Bob","Maths":72,"Science":68,"English":74}, 5 {"StudentID":3,"Name":"Charlie","Maths":90,"Science":95,"English":88}, 6 {"StudentID":4,"Name":"David","Maths":60,"Science":55,"English":65} 7] 8 9 # Filter students with Maths > 80 10 filtered = [s for s in students if s["Maths"] > 80] 11 12 # Calculate total and average marks 13 stats = {} 14 for s in filtered: 15 total = s["Maths"] + s["Science"] + s["English"] 16 average = total / 3 17 stats[s["Name"]] = {"Total": total, "Average": average} 18 19 print("Students with Maths > 80:") 20 for s in filtered: 21 print(s) 22 23 print("\nTotal & Average Marks per Student:") 24 for name, stat in stats.items(): 25 print(name, stat)</pre>	Sales > 50,000: <pre>{'SaleID': 101, 'Product': 'Laptop', 'Amount': 75000, 'Customer': 'A'} {'SaleID': 104, 'Product': 'Laptop', 'Amount': 120000, 'Customer': 'D'} {'SaleID': 105, 'Product': 'Mouse', 'Amount': 60000, 'Customer': 'E'}</pre> Total & Average Sales per Product: <pre>Laptop {'total': 195000, 'average': 97500.0} Mouse {'total': 60000, 'average': 60000.0}</pre> === Code Execution Successful ===

4. Create a CSV file named **products.csv** with columns **ProductID**, **ProductName**, **Category**, **Stock**. Load the data, filter products with stock less than 50, group by Category, and calculate the total and average stock per category.

main.py	Output	Clear
<pre>1 # Exp-13 Q4: Products Stock Analysis (No pandas) 2 from collections import defaultdict 3 products = [4 {"ProductID":1,"ProductName":"Pen","Category":"Stationery","Stock":100}, 5 {"ProductID":2,"ProductName":"Notebook","Category":"Stationery","Stock":30}, 6 {"ProductID":3,"ProductName":"Eraser","Category":"Stationery","Stock":25}, 7 {"ProductID":4,"ProductName":"Marker","Category":"Stationery","Stock":80}, 8 {"ProductID":5,"ProductName":"Book","Category":"Book","Stock":40} 9] 10 # Filter products with Stock < 50 11 filtered = [p for p in products if p["Stock"] < 50] 12 # Group by Category and calculate total & average stock 13 grouped = defaultdict(list) 14 for p in filtered: 15 grouped[p["Category"]].append(p["Stock"]) 16 stats = {cat: {"total": sum(v), "average": sum(v)/len(v)} for cat,v in grouped.items()} 17 print("Products with Stock < 50:") 18 for p in filtered: 19 print(p) 20 print("\nTotal & Average Stock per Category:") 21 for cat, stat in stats.items(): 22 print(cat, stat)</pre>	Products with Stock < 50: <pre>{'ProductID': 2, 'ProductName': 'Notebook', 'Category': 'Stationery', 'Stock': 30} {'ProductID': 3, 'ProductName': 'Eraser', 'Category': 'Stationery', 'Stock': 25} {'ProductID': 5, 'ProductName': 'Book', 'Category': 'Book', 'Stock': 40}</pre> Total & Average Stock per Category: <pre>Stationery {'total': 55, 'average': 27.5} Book {'total': 40, 'average': 40.0}</pre> === Code Execution Successful ===	

Name : Srushti Patil

Roll No.: C-04

Software Proficiency Program II (Python Programming)

5. Create a CSV file named **transactions.csv** with columns **TransactionID**, **Customer**, **Amount**, **Date**. Load the data, filter transactions with amount greater than 1000, group by Customer, and calculate total and average spending per customer.

main.py	Output
<pre>1 # Exp-13 Q5: Transactions Analysis 2 from collections import defaultdict 3 transactions = [4 {"TransactionID":1,"Customer":"A","Amount":1200,"Date":"2025-10-01"}, 5 {"TransactionID":2,"Customer":"B","Amount":800,"Date":"2025-10-02"}, 6 {"TransactionID":3,"Customer":"A","Amount":1500,"Date":"2025-10-03"}, 7 {"TransactionID":4,"Customer":"C","Amount":2000,"Date":"2025-10-04"}, 8 {"TransactionID":5,"Customer":"B","Amount":500,"Date":"2025-10-05"} 9] 10 # Filter transactions > 1000 11 filtered = [t for t in transactions if t["Amount"] > 1000] 12 # Group by Customer and calculate total & average 13 grouped = defaultdict(list) 14 for t in filtered: 15 grouped[t["Customer"]].append(t["Amount"]) 16 17 stats = {c: {"total": sum(v), "average": sum(v)/len(v)} for c,v in grouped.items() 18 ()} 19 print("Transactions > 1000:") 20 for t in filtered: 21 print(t) 22 print("\nTotal & Average Spending per Customer:") 23 for customer, stat in stats.items(): 24 print(customer, stat)</pre>	<pre>Transactions > 1000: {'TransactionID': 1, 'Customer': 'A', 'Amount': 1200, 'Date': '2025-10-01'} {'TransactionID': 3, 'Customer': 'A', 'Amount': 1500, 'Date': '2025-10-03'} {'TransactionID': 4, 'Customer': 'C', 'Amount': 2000, 'Date': '2025-10-04'} Total & Average Spending per Customer: A {'total': 2700, 'average': 1350.0} C {'total': 2000, 'average': 2000.0} === Code Execution Successful ===</pre>

6. Create a CSV file named **employee_salary.csv** with columns **EmpID**, **Name**, **Department**, **Salary**. Load the data, filter employees with salary above 60,000, group by Department, and calculate mean, max, and min salary per department.

main.py	Output
<pre>1 # Exp-13 Q6: Employee Salary Analysis 2 from collections import defaultdict 3 employees = [4 {"EmpID":1,"Name":"Alice","Department":"HR","Salary":50000}, 5 {"EmpID":2,"Name":"Bob","Department":"IT","Salary":70000}, 6 {"EmpID":3,"Name":"Charlie","Department":"Finance","Salary":60000}, 7 {"EmpID":4,"Name":"David","Department":"HR","Salary":45000}, 8 {"EmpID":5,"Name":"Eva","Department":"IT","Salary":80000} 9] 10 # Filter salary > 60000 11 filtered = [e for e in employees if e["Salary"] > 60000] 12 # Group by Department and calculate mean, max, min 13 grouped = defaultdict(list) 14 for e in filtered: 15 grouped[e["Department"]].append(e["Salary"]) 16 17 stats = {d: {"mean": sum(v)/len(v), "max": max(v), "min": min(v)} for d,v in 18 grouped.items()} 19 print("Employees with Salary > 60000:") 20 for e in filtered: 21 print(e) 22 print("\nSalary Stats per Department:") 23 for dept, stat in stats.items(): 24 print(dept, stat)</pre>	<pre>Employees with Salary > 60000: {'EmpID': 2, 'Name': 'Bob', 'Department': 'IT', 'Salary': 70000} {'EmpID': 5, 'Name': 'Eva', 'Department': 'IT', 'Salary': 80000} Salary Stats per Department: IT {'mean': 75000.0, 'max': 80000, 'min': 70000} === Code Execution Successful ===</pre>

Name : Srushti Patil

Roll No.: C-04

Software Proficiency Program II (Python Programming)

7. **Create a CSV file named orders.csv with columns OrderID, Customer, Product, Quantity, OrderDate. Load the data, filter orders with quantity greater than 5, group by Product, and calculate total quantity sold per product.**

main.py	Output
<pre>1 # Exp-13 Q7: Orders Analysis 2 from collections import defaultdict 3 orders = [4 {"OrderID":1,"Customer":"A","Product":"Pen","Quantity":10}, 5 {"OrderID":2,"Customer":"B","Product":"Notebook","Quantity":3}, 6 {"OrderID":3,"Customer":"A","Product":"Pen","Quantity":7}, 7 {"OrderID":4,"Customer":"C","Product":"Marker","Quantity":6} 8] 9 # Filter quantity > 5 10 filtered = [o for o in orders if o["Quantity"] > 5] 11 # Group by Product and calculate total quantity 12 grouped = defaultdict(int) 13 for o in filtered: 14 grouped[o["Product"]] += o["Quantity"] 15 print("Orders with Quantity > 5:") 16 for o in filtered: 17 print(o) 18 print("\nTotal Quantity Sold per Product:") 19 for product, qty in grouped.items(): 20 print(product, qty)</pre>	Orders with Quantity > 5: <pre>{'OrderID': 1, 'Customer': 'A', 'Product': 'Pen', 'Quantity': 10} {'OrderID': 3, 'Customer': 'A', 'Product': 'Pen', 'Quantity': 7} {'OrderID': 4, 'Customer': 'C', 'Product': 'Marker', 'Quantity': 6}</pre> Total Quantity Sold per Product: Pen 17 Marker 6 === Code Execution Successful ===

8. **Create a CSV file named movies.csv with columns MovieID, Title, Genre, Rating. Load the data, filter movies with rating above 8, group by Genre, and calculate average, maximum, and minimum rating per genre.**

main.py	Output
<pre>1 # Exp-13 Q8: Movies Analysis 2 from collections import defaultdict 3 movies = [4 {"MovieID":1,"Title":"Movie1","Genre":"Action","Rating":8.5}, 5 {"MovieID":2,"Title":"Movie2","Genre":"Drama","Rating":7.9}, 6 {"MovieID":3,"Title":"Movie3","Genre":"Action","Rating":9.0}, 7 {"MovieID":4,"Title":"Movie4","Genre":"Comedy","Rating":8.2} 8] 9 # Filter rating > 8 10 filtered = [m for m in movies if m["Rating"] > 8] 11 # Group by Genre and calculate mean, max, min 12 grouped = defaultdict(list) 13 for m in filtered: 14 grouped[m["Genre"]].append(m["Rating"]) 15 stats = {g: {"mean": sum(v)/len(v), "max": max(v), "min": min(v)} for g 16 ,v in grouped.items()} 17 print("Movies with Rating > 8:") 18 for m in filtered: 19 print(m) 20 for genre, stat in stats.items(): 21 print(genre, stat)</pre>	Movies with Rating > 8: <pre>{'MovieID': 1, 'Title': 'Movie1', 'Genre': 'Action', 'Rating': 8.5} {'MovieID': 3, 'Title': 'Movie3', 'Genre': 'Action', 'Rating': 9.0} {'MovieID': 4, 'Title': 'Movie4', 'Genre': 'Comedy', 'Rating': 8.2}</pre> Rating Stats by Genre: Action {'mean': 8.75, 'max': 9.0, 'min': 8.5} Comedy {'mean': 8.2, 'max': 8.2, 'min': 8.2} === Code Execution Successful ===

Name : Srushti Patil

Roll No.: C-04

Software Proficiency Program II (Python Programming)

9. Create a CSV file named **weather.csv** with columns **Date**, **City**, **Temperature**, **Humidity**. Load the data, filter days with temperature above 35°C, group by City, and calculate maximum, minimum, and average temperature for each city.

main.py	Output
<pre>1 from collections import defaultdict 2 weather = [3 {"Date": "2025-10-01", "City": "CityA", "Temperature": 36, "Humidity": 50}, 4 {"Date": "2025-10-02", "City": "CityB", "Temperature": 34, "Humidity": 60}, 5 {"Date": "2025-10-03", "City": "CityA", "Temperature": 38, "Humidity": 55} 6] 7 # Filter temperature > 35 8 filtered = [w for w in weather if w["Temperature"] > 35] 9 # Group by City and calculate max, min, average 10 grouped = defaultdict(list) 11 for w in filtered: 12 grouped[w["City"]].append(w["Temperature"]) 13 stats = {c: {"max": max(v), "min": min(v), "average": sum(v)/len(v)} 14 for c,v in grouped.items()} 15 print("Days with Temperature > 35:") 16 for w in filtered: 17 print(w) 18 print("\nTemperature Stats per City:") 19 for city, stat in stats.items(): 20 print(city, stat)</pre>	<pre>Days with Temperature > 35: {'Date': '2025-10-01', 'City': 'CityA', 'Temperature': 36, 'Humidity': 50} {'Date': '2025-10-03', 'City': 'CityA', 'Temperature': 38, 'Humidity': 55} Temperature Stats per City: CityA {'max': 38, 'min': 36, 'average': 37.0} === Code Execution Successful ===</pre>

10. Create a CSV file named **bank transactions.csv** with columns **TransactionID**, **Customer**, **Type**, **Amount**. Load the data, filter deposits, group by Customer, and calculate total, maximum, and average deposit amount per customer.

<pre>1 # Exp-13 Q10: Bank Transactions 2 from collections import defaultdict 3 bank = [4 {"TransactionID": 1, "Customer": "A", "Type": "Deposit", "Amount": 1500}, 5 {"TransactionID": 2, "Customer": "B", "Type": "Withdrawal", "Amount": 500}, 6 {"TransactionID": 3, "Customer": "A", "Type": "Deposit", "Amount": 2000}, 7 {"TransactionID": 4, "Customer": "C", "Type": "Deposit", "Amount": 1200} 8] 9 # Filter deposits only 10 filtered = [b for b in bank if b["Type"] == "Deposit"] 11 # Group by Customer and calculate total, max, average 12 grouped = defaultdict(list) 13 for b in filtered: 14 grouped[b["Customer"]].append(b["Amount"]) 15 stats = {c: {"total": sum(v), "max": max(v), "average": sum(v)/len(v)} 16 for c,v in grouped.items()} 17 print("Deposits Only:") 18 for b in filtered: 19 print(b) 20 print("\nDeposit Stats per Customer:") 21 for customer, stat in stats.items(): 22 print(customer, stat)</pre>	<pre>Deposits Only: {'TransactionID': 1, 'Customer': 'A', 'Type': 'Deposit', 'Amount': 1500} {'TransactionID': 3, 'Customer': 'A', 'Type': 'Deposit', 'Amount': 2000} {'TransactionID': 4, 'Customer': 'C', 'Type': 'Deposit', 'Amount': 1200} Deposit Stats per Customer: A {'total': 3500, 'max': 2000, 'average': 1750.0} C {'total': 1200, 'max': 1200, 'average': 1200.0} === Code Execution Successful ===</pre>

Name : Srushti Patil

Roll No.: C-04

Software Proficiency Program II (Python Programming)

- 11. Create a CSV file named hospital.csv with columns PatientID, Name, Department, Admission Date. Load the data, filter patients admitted in the last month, group by Department, and calculate the total number of patients per department.**

main.py	Output
<pre>1 from collections import defaultdict 2 patients=[{"PatientID":1,"Name":"Alice","Department": : "Cardiology","AdmissionDate":"2025-09-15"}, 3 {"PatientID":2,"Name":"Bob","Department": "Neurology" , "AdmissionDate": "2025-10-10"}, 4 {"PatientID":3,"Name":"Charlie","Department" : "Cardiology","AdmissionDate":"2025-10-05"}, 5 {"PatientID":4,"Name":"David","Department" : "Orthopedics","AdmissionDate":"2025-09-25"}] 6 filtered=[p for p in patients if p["AdmissionDate"]>="2025-09 -30"] 7 grouped=defaultdict(int) 8 for p in filtered: grouped[p["Department"]]+=1 9 print("Filtered Patients:") 10 for p in filtered: print(p) 11 print("\nTotal Patients per Department:") 12 for dept, count in grouped.items(): print(dept, count)</pre>	Filtered Patients: <pre>{'PatientID': 2, 'Name': 'Bob', 'Department': 'Neurology', 'AdmissionDate': '2025-10-10'} {'PatientID': 3, 'Name': 'Charlie', 'Department': 'Cardiology', 'AdmissionDate': '2025-10-05'}</pre> Total Patients per Department: <pre>Neurology 1 Cardiology 1</pre> === Code Execution Successful ===

- 12. Create a CSV file named online store.csv with columns OrderID, Customer, Product, Amount. Load the data, filter orders with amount greater than 200, group by Customer, and calculate average, total, and maximum order amount per customer.**

main.py	Output
<pre>1 from collections import defaultdict 2 orders=[{"OrderID":1,"Customer":"A","Product":"Pen","Amount" :150}, 3 {"OrderID":2,"Customer":"B","Product":"Notebook" , "Amount":250}, 4 {"OrderID":3,"Customer":"A","Product":"Marker" , "Amount":300}, 5 {"OrderID":4,"Customer":"C","Product":"Book","Amount" :100}] 6 filtered=[o for o in orders if o["Amount"]>200] 7 grouped=defaultdict(list) 8 for o in filtered: grouped[o["Customer"]].append(o["Amount"]) 9 stats={c:{ "total":sum(v),"average":sum(v)/len(v),"max":max(v)} for c,v in grouped.items()} 10 print("Orders with Amount > 200:") 11 for o in filtered: print(o) 12 print("\nStats per Customer:") 13 for c, stat in stats.items(): print(c, stat)</pre>	Orders with Amount > 200: <pre>{'OrderID': 2, 'Customer': 'B', 'Product': 'Notebook', 'Amount': 250} {'OrderID': 3, 'Customer': 'A', 'Product': 'Marker', 'Amount': 300}</pre> Stats per Customer: <pre>B {'total': 250, 'average': 250.0, 'max': 250} A {'total': 300, 'average': 300.0, 'max': 300}</pre> === Code Execution Successful ===

Name : Srushti Patil

Roll No.: C-04

Software Proficiency Program II (Python Programming)

- 13. Create a CSV file named flights.csv with columns FlightID, Airline, Source, Destination, Delay. Load the data, filter flights with delay above 30 minutes, group by Airline, and calculate total flights, average, and maximum delay per airline.**

main.py	Output
<pre>1 from collections import defaultdict 2 flights=[{"FlightID":1,"Airline":"AirA","Source":"CityX", 3 "Destination":"CityY","Delay":45}, 4 {"FlightID":2,"Airline":"AirB","Source":"CityY", 5 "Destination":"CityZ","Delay":20}, 6 {"FlightID":3,"Airline":"AirA","Source":"CityX", 7 "Destination":"CityZ","Delay":35}, 8 {"FlightID":4,"Airline":"AirC","Source":"CityZ", 9 "Destination":"CityX","Delay":50}] 10 filtered=[f for f in flights if f["Delay"]>30] 11 grouped=defaultdict(list) 12 for f in filtered: grouped[f["Airline"]].append(f["Delay"]) 13 stats={a:{'total_flights':len(v),'average':sum(v)/len(v),'max': 14 :max(v)} for a,v in grouped.items()} 15 print("Flights with Delay > 30:") 16 for f in filtered: print(f) 17 print("\nDelay Stats per Airline:") 18 for a, stat in stats.items(): print(a, stat)</pre>	Flights with Delay > 30: <pre>{'FlightID': 1, 'Airline': 'AirA', 'Source': 'CityX', 'Destination': 'CityY', 'Delay': 45} {'FlightID': 3, 'Airline': 'AirA', 'Source': 'CityX', 'Destination': 'CityZ', 'Delay': 35} {'FlightID': 4, 'Airline': 'AirC', 'Source': 'CityZ', 'Destination': 'CityX', 'Delay': 50}</pre> Delay Stats per Airline: <pre>AirA {'total_flights': 2, 'average': 40.0, 'max': 45} AirC {'total_flights': 1, 'average': 50.0, 'max': 50}</pre> === Code Execution Successful ===

- 14. Create a CSV file named inventory.csv with columns ProductID, ProductName, Supplier, Stock. Load the data, filter out-of-stock products, group by Supplier, and calculate total out-of-stock products per supplier.**

main.py	Output
<pre>1 from collections import defaultdict 2 inventory=[{"ProductID":1,"ProductName":"Pen","Supplier":"S1", 3 "Stock":0}, 4 {"ProductID":2,"ProductName":"Notebook","Supplier": 5 "S2","Stock":50}, 6 {"ProductID":3,"ProductName":"Eraser","Supplier": 7 "S1","Stock":0}, 8 {"ProductID":4,"ProductName":"Marker","Supplier": 9 "S3","Stock":30}] 10 filtered=[i for i in inventory if i["Stock"]==0] 11 grouped=defaultdict(int) 12 for i in filtered: grouped[i["Supplier"]]+=1 13 print("Out-of-Stock Products:") 14 for i in filtered: print(i) 15 print("\nOut-of-Stock Count per Supplier:") 16 for s, count in grouped.items(): print(s, count)</pre>	Out-of-Stock Products: <pre>{'ProductID': 1, 'ProductName': 'Pen', 'Supplier': 'S1', 'Stock': 0} {'ProductID': 3, 'ProductName': 'Eraser', 'Supplier': 'S1', 'Stock': 0}</pre> Out-of-Stock Count per Supplier: <pre>S1 2</pre> === Code Execution Successful ===

Name : Srushti Patil

Roll No.: C-04

Software Proficiency Program II (Python Programming)

- 15. Create a CSV file named `ecommerce.csv` with columns `OrderID`, `Category`, `SubCategory`, `Sales`. Load the data, filter sales above 500, group by `Category` and `SubCategory`, and calculate total, average, and maximum sales per combination.**

main.py	Output
<pre>1 from collections import defaultdict 2 sales=[{"OrderID":1,"Category":"Electronics","SubCategory": : "Mobile","Sales":600}, 3 {"OrderID":2,"Category":"Electronics","SubCategory" : "Laptop","Sales":1200}, 4 {"OrderID":3,"Category":"Furniture","SubCategory" : "Chair","Sales":300}, 5 {"OrderID":4,"Category":"Electronics","SubCategory" : "Mobile","Sales":700}] 6 filtered=[s for s in sales if s["Sales"]>500] 7 grouped=defaultdict(list) 8 for s in filtered: grouped[(s["Category"],s["SubCategory"])] .append(s["Sales"]) 9 stats={k:{"total":sum(v),"average":sum(v)/len(v),"max":max(v)} for k,v in grouped.items()} 10 print("Sales > 500:") 11 for s in filtered: print(s) 12 print("\nSales Stats per Category & SubCategory:")for k, stat in stats.items(): print(k, stat)</pre>	<pre>Sales > 500: {'OrderID': 1, 'Category': 'Electronics', 'SubCategory': 'Mobile', 'Sales': 600} {'OrderID': 2, 'Category': 'Electronics', 'SubCategory': 'Laptop', 'Sales': 1200} {'OrderID': 4, 'Category': 'Electronics', 'SubCategory': 'Mobile', 'Sales': 700} Sales Stats per Category & SubCategory: ('Electronics', 'Mobile') {'total': 1300, 'average': 650.0, 'max': 700} ('Electronics', 'Laptop') {'total': 1200, 'average': 1200.0, 'max': 1200} === Code Execution Successful ===</pre>

- 16. Create a CSV file named `employee_performance.csv` with columns `EmpID`, `Name`, `Department`, `Rating`, `Bonus`. Load the data, filter employees with `Rating` above 4.0, group by `Department`, and calculate average rating and total bonus per department.**

main.py	Output
<pre>1 from collections import defaultdict 2 data=[{"ID":1,"Type":"A","Value":100}, 3 {"ID":2,"Type":"B","Value":200}, 4 {"ID":3,"Type":"A","Value":150}, 5 {"ID":4,"Type":"B","Value":50}] 6 filtered=[d for d in data if d["Value"]>100] 7 grouped=defaultdict(list) 8 for d in filtered: grouped[d["Type"]].append(d["Value"]) 9 stats={t:{"total":sum(v),"average":sum(v)/len(v)} for t,v in grouped.items()} 10 print("Filtered Data:") 11 for d in filtered: print(d) 12 print("\nStats per Type:") 13 for t, stat in stats.items(): print(t, stat)</pre>	<pre>Filtered Data: {'ID': 2, 'Type': 'B', 'Value': 200} {'ID': 3, 'Type': 'A', 'Value': 150} Stats per Type: B {'total': 200, 'average': 200.0} A {'total': 150, 'average': 150.0} === Code Execution Successful ===</pre>